

React Derinleşme

useEffect, Koşullu Render, Form ve ToDo

Session Akışı

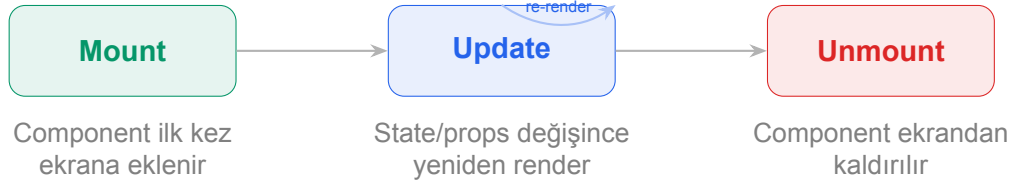
Süre	Konu	Anahtar Kavramlar
0-25 dk	useEffect ve Lifecycle	useEffect, dependency array, cleanup
25-50 dk	Conditional & List Rendering	ternary, &&, map, key
50-75 dk	Form Handling ve Validation	controlled input, onSubmit, validation
75-90 dk	Pratik: Mini ToDo App	useState, useEffect, form, CRUD

1 useEffect ve Lifecycle

 0–25 dakika

1.1 Component Lifecycle Nedir?

Bir React component'inin üç temel yaşam evresi vardır:



useEffect Nedir?

`useEffect`, component'in **yan etkilerini** (side effects) yönetmek için kullanılan React Hook'udur. Yan etki = render dışında olan her şey:

- API'den veri çekme (fetch)
- DOM manipülasyonu
- Timer (setTimeout, setInterval)
- Event listener ekleme/kaldırma
- localStorage okuma/yazma

1.2 useEffect Söz Dizimi

useEffect Formülü

```
import { useEffect } from "react";

useEffect(() => {
  // Yan etki burada calisir

  return () => {
    // Cleanup (temizlik) --- opsiyonel
  };
}, [bagimlilikler]); // dependency array
```

1.2.1 Dependency Array Davranışları

Kullanım	Ne Zaman Çalışır?	Benzetme
<code>useEffect(() => {}, [])</code>	Sadece ilk render'da (mount)	<i>componentDidMount</i>
<code>useEffect(() => {}, [a, b])</code>	İlk render + a veya b değişince	<i>componentDidUpdate</i>
<code>useEffect(() => {})</code>	Her render'da (genellikle istemezsiniz!)	—
<code>return () => {}</code>	Unmount veya effect tekrar çalışmadan önce	<i>componentWillUnmount</i>

Örnek 1 — Sayfa Başlığını Güncelleme

```
import { useState, useEffect } from "react";

function Counter() {
  const [count, setCount] = useState(0);

  // count değişince sayfa başlığını güncelle
  useEffect(() => {
    document.title = `Count: ${count}`;
  }, [count]);

  return (
    <div>
      <p>{count}</p>
      <button onClick={() => setCount(count + 1)}>
        +1
      </button>
    </div>
  );
}
```

Örnek 2 — API'den Veri Çekme (En Yaygın Kullanım)

```
import { useState, useEffect } from "react";

function UserList() {
  const [users, setUsers] = useState([]);
  const [loading, setLoading] = useState(true);

  useEffect(() => {
    async function fetchUsers() {
      const res = await fetch(
        "https://jsonplaceholder.typicode.com/users"
      );
      const data = await res.json();
      setUsers(data);
      setLoading(false);
    }

    fetchUsers();
  }, []); // bos dizi = sadece mount'ta calis

  if (loading) return <p>Loading...</p>;

  return (
    <ul>
      {users.map(user => (
        <li key={user.id}>{user.name}</li>
      ))}
    </ul>
  );
}
```

1.3 Cleanup (Temizlik) Fonksiyonu

Bazı yan etkiler temizlenmezse **memory leak** (bellek sızıntısı) oluşur. Örneğin timer veya event listener eklediysek, component kaldırılınca bunları temizlemeliyiz.

Cleanup — Timer Örneği

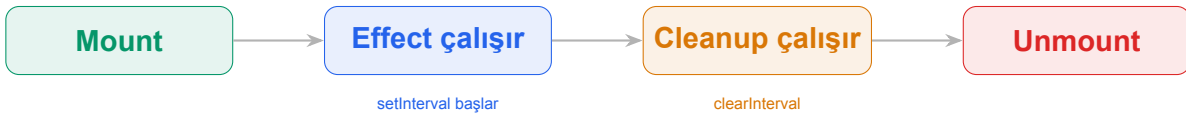
```
import { useState, useEffect } from "react";

function Timer() {
  const [seconds, setSeconds] = useState(0);

  useEffect(() => {
    const interval = setInterval(() => {
      setSeconds(prev => prev + 1);
    }, 1000);

    // Cleanup: component unmount olunca timer'i durdur
    return () => clearInterval(interval);
  }, []);

  return <p>Elapsed: {seconds}s</p>;
}
```



⚠ Sık Yapılan Hatalar

1. **Dependency array unutmak:** Effect her render'da çalışır, sonsuz döngüye girebilir.
2. **Bağımlılık eksik bırakmak:** Effect eski (stale) değerlerle çalışır.
3. **useEffect içinde doğrudan async fonksiyon:** `useEffect(async () => ...)` **yazılmaz**. İçeride ayrı bir async fonksiyon tanımlayıp çağırın.

2 📌 Conditional ve List Rendering

🕒 25–50 dakika

2.1 Conditional Rendering (Koşullu Gösterim)

React'ta bir component'in ne göstereceğini koşullara göre değiştirebiliriz. Bunun birkaç yolu var:

2.1.1 Yöntem 1 — if / else ile Erken Return

if/else — Erken Return

```
function Dashboard({ isLoggedIn }) {  
  if (!isLoggedIn) {  
    return <p>Please login first.</p>;  
  }  
  
  return <h1>Welcome to Dashboard!</h1>;  
}
```

2.1.2 Yöntem 2 — Ternary Operatör (? :)

Ternary ile Koşullu Render

```
function Greeting({ isLoggedIn, userName }) {  
  return (  
    <div>  
      {isLoggedIn  
        ? <h1>Welcome, {userName}!</h1>  
        : <h1>Please sign in.</h1>  
      }  
    </div>  
  );  
}
```

2.1.3 Yöntem 3 — && (Logical AND)

Sadece bir koşul sağlandığında göstermek istiyorsak:

&& ile Koşullu Render

```
function Notifications({ count }) {
  return (
    <div>
      <h1>Home</h1>
      {count > 0 && (
        <span className="badge">{count} new notifications</span>
      )}
    </div>
  );
}
```

Yöntem	Ne Zaman?	Örnek
<code>if / return</code>	Tamamen farklı UI döndürme	Loading / Error / Data
<code>? :</code>	İki alternatif arasında seçim	Login / Logout butonu
<code>&&</code>	Bir şeyi göster veya hiç gösterme	Badge, uyarı mesajı

! && ile Sıfır Tuzağı

`{count && <span>{count}</span>}` yazdığınızda, `count` sıfırsa ekranda 0 görünür (false değil!). Çözüm: `{count > 0 && ...}` şeklinde açıkça boolean karşılaştırma yapın.

2.2 Gerçekçi Örnek — Loading / Error / Data Patterni

API çağrılarında en yaygın conditional rendering kalıbı:

Loading / Error / Data

```
function ProductList() {
  const [products, setProducts] = useState([]);
  const [loading, setLoading] = useState(true);
  const [error, setError] = useState(null);

  useEffect(() => {
    async function fetchProducts() {
      try {
        const res = await fetch("/api/products");
        if (!res.ok) throw new Error("Failed to fetch");
        const data = await res.json();
        setProducts(data);
      } catch (err) {
        setError(err.message);
      } finally {
        setLoading(false);
      }
    }
    fetchProducts();
  }, []);

  if (loading) return <p>Loading...</p>;
  if (error) return <p className="error">{error}</p>;
  if (products.length === 0) return <p>No products found.</p>;

  return (
    <ul>
      {products.map(p => (
        <li key={p.id}>{p.name} - {p.price} TL</li>
      ))}
    </ul>
  );
}
```

2.3 List Rendering ve key Prop'u

2.3.1 map ile Liste Render

React'ta bir diziyi UI'a dönüştürmek için map kullanılır:

Liste Render Örneği

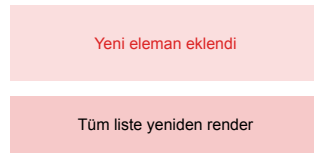
```
const fruits = ["Apple", "Banana", "Cherry"];

function FruitList() {
  return (
    <ul>
      {fruits.map((fruit, index) => (
        <li key={index}>{fruit}</li>
      ))}
    </ul>
  );
}
```

2.3.2 key Neden Önemli?

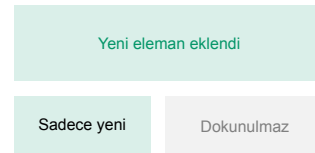
React, listedeki hangi elemanın eklendiğini, silindiğini veya değiştiğini anlamak için `key` prop'unu kullanır. `key` olmazsa React tüm listeyi baştan render eder.

key yok / index key:



⚠️ Yavaş + state kaybı riski

Benzersiz key:



✅ Hızlı + state korunur

⚠️ key Kuralları

- `key` **benzersiz ve sabit** olmalıdır (veritabanı id idealdir).
- `index` son çare olarak kullanılabilir — sıralama veya silme yapılan listelerde **so-run yaratır**.
- `key` bir prop değildir; component içinden `props.key` ile erişilemez.
- `key` sadece kardeş elemanlar arasında benzersiz olmalıdır, global benzersizlik gerekmez.

Doğru key Kullanımı

```
const users = [
  { id: 1, name: "Ali" },
  { id: 2, name: "Veli" },
  { id: 3, name: "Zeynep" },
];

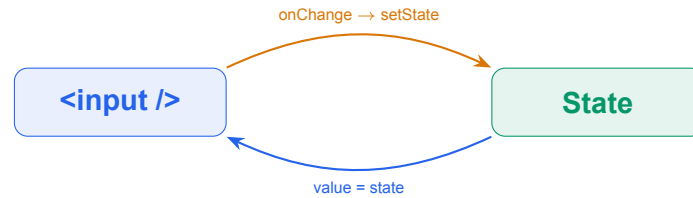
function UserList() {
  return (
    <ul>
      {users.map(user => (
        <li key={user.id}>{user.name}</li>
      ))}
    </ul>
  );
}
```

3 Form Handling ve Validation

 50–75 dakika

3.1 Controlled Component (Hatırlatma)

Session 3'te öğrendiğimiz gibi, React'ta input'ların değeri **state'te** tutulur. Bu yaklaşıma **controlled component** denir.



3.2 Form Submit

Form gönderimi için `onSubmit` event'i kullanılır. `e.preventDefault()` ile sayfanın yenilenmesini önleriz.

Form Submit Örneği

```
import { useState } from "react";

function LoginForm() {
  const [email, setEmail] = useState("");
  const [password, setPassword] = useState("");

  const handleSubmit = (e) => {
    e.preventDefault(); // sayfa yenilenmesin

    console.log("Email:", email);
    console.log("Password:", password);
    // API çağrısı yapılabilir
  };

  return (
    <form onSubmit={handleSubmit}>
      <label htmlFor="email">Email:</label>
      <input
        id="email"
        type="email"
        value={email}
        onChange={(e) => setEmail(e.target.value)}
      />

      <label htmlFor="password">Password:</label>
      <input
        id="password"
        type="password"
        value={password}
        onChange={(e) => setPassword(e.target.value)}
      />

      <button type="submit">Login</button>
    </form>
  );
}
```

3.3 Birden Fazla Input — Tek State Objesi

Çok sayıda input varsa her biri için ayrı state yerine tek bir **obje state** kullanmak daha pratiktir:

Obje State ile Çoklu Input

```
import { useState } from "react";

function RegisterForm() {
  const [form, setForm] = useState({
    name: "",
    email: "",
    password: "",
  });

  const handleChange = (e) => {
    const { name, value } = e.target;
    setForm(prev => ({
      ...prev,          // önceki degerleri koru
      [name]: value,    // sadece degisen alani guncelle
    }));
  };

  const handleSubmit = (e) => {
    e.preventDefault();
    console.log("Form data:", form);
  };

  return (
    <form onSubmit={handleSubmit}>
      <input name="name" value={form.name}
        onChange={handleChange} placeholder="Name" />
      <input name="email" value={form.email}
        onChange={handleChange} placeholder="Email" />
      <input name="password" value={form.password}
        onChange={handleChange} placeholder="Password"
        type="password" />
      <button type="submit">Register</button>
    </form>
  );
}
```

💡 Computed Property Name

[name]: value söz dizimi JavaScript'in **computed property name** özelliğidir. name değişkeninin değeri ne ise o key güncellenir. Session 2'deki destructuring ve spread bilgisi burada devreye giriyor.

3.4 Basic Validation (Temel Doğrulama)

Form gönderilmeden önce verilerin geçerliliğini kontrol etmemiz gerekir. En basit yaklaşım: submit anında kontrol et, hata varsa göster.

Temel Validation

```
import { useState } from "react";

function ContactForm() {
  const [name, setName] = useState("");
  const [email, setEmail] = useState("");
  const [errors, setErrors] = useState({});

  const validate = () => {
    const newErrors = {};
    if (!name.trim()) newErrors.name = "Name is required";
    if (!email.trim()) {
      newErrors.email = "Email is required";
    } else if (!email.includes("@")) {
      newErrors.email = "Invalid email";
    }
    return newErrors;
  };

  const handleSubmit = (e) => {
    e.preventDefault();
    const validationErrors = validate();

    if (Object.keys(validationErrors).length > 0) {
      setErrors(validationErrors);
      return; // göndermeden dur
    }

    setErrors({});
    console.log("Form submitted:", { name, email });
  };

  return (
    <form onSubmit={handleSubmit}>
      <input value={name}
        onChange={(e) => setName(e.target.value)}
        placeholder="Name" />
      {errors.name && (
        <span className="error">{errors.name}</span>
      )}

      <input value={email}
        onChange={(e) => setEmail(e.target.value)}
        placeholder="Email" />
      {errors.email && (
        <span className="error">{errors.email}</span>
      )}

      <button type="submit">Send</button>
    </form>
  );
}
```

Yaklaşım	Açıklama
Submit validation	Form gönderildiğinde kontrol et. Basit ve anlaşılır.
onChange validation	Her tuş vuruşunda kontrol et. Anlık geri bildirim ama daha fazla kod.
onBlur validation	Input'tan çıkınca kontrol et. İkisinin arası — kullanıcı dostu.

4 Pratik: Mini ToDo Uygulaması

 75–90 dakika

Pratik Hedef

Bu session'da öğrendiğimiz her şeyi birleştirerek bir **ToDo uygulaması** yazacağız:

1. Yeni görev ekleme (form + validation)
2. Görev listesi render etme (map + key)
3. Görevi tamamlandı olarak işaretleme (conditional rendering)
4. Görevi silme (filter ile state güncelleme)
5. Kalan görev sayısını gösterme

TodoApp.jsx

```
import { useState } from "react";

function TodoApp() {
  const [todos, setTodos] = useState([]);
  const [text, setText] = useState("");
  const [error, setError] = useState("");

  // --- Ekleme ---
  const handleAdd = (e) => {
    e.preventDefault();

    if (!text.trim()) {
      setError("Task cannot be empty!");
      return;
    }

    const newTodo = {
      id: Date.now(),
      text: text.trim(),
      completed: false,
    };

    setTodos([...todos, newTodo]);
    setText("");
    setError("");
  };

  // --- Tamamlandi/geri al ---
  const toggleTodo = (id) => {
    setTodos(
      todos.map(todo =>
        todo.id === id
          ? { ...todo, completed: !todo.completed }
          : todo
      )
    );
  };

  // --- Silme ---
  const deleteTodo = (id) => {
    setTodos(todos.filter(todo => todo.id !== id));
  };

  const remaining = todos.filter(t => !t.completed).length;
```

TodoApp.jsx (devam — return kısmı)

```

return (
  <div style={{ maxWidth: 500, margin: "40px auto" }}>
    <h1>My Todos</h1>

    { /* --- Form --- */ }
    <form onSubmit={handleAdd}>
      <input
        type="text"
        value={text}
        onChange={ (e) => setText(e.target.value) }
        placeholder="What needs to be done?"
      />
      <button type="submit">Add</button>
    </form>
    {error && <p style={{ color: "red" }}>{error}</p>}

    { /* --- Liste --- */ }
    <ul style={{ listStyle: "none", padding: 0 }}>
      {todos.map(todo => (
        <li key={todo.id}
          style={{
            textDecoration: todo.completed
              ? "line-through" : "none",
            color: todo.completed
              ? "#94a3b8" : "#1e293b",
          }}>

          <input type="checkbox"
            checked={todo.completed}
            onChange={() => toggleTodo(todo.id)} />

          <span>{todo.text}</span>

          <button onClick={() => deleteTodo(todo.id)}>
            Delete
          </button>
        </li>
      ) ) }
    </ul>

    { /* --- Özet --- */ }
    {todos.length > 0 && (
      <p>{remaining} task(s) remaining</p>
    ) }
  </div>
);
}

export default TodoApp;

```

💡 Bu Kodda Neler Kullandık?

- `useState` — 3 farklı state (`todos`, `text`, `error`)
- `onSubmit + e.preventDefault()` — form gönderimi
- `validation` — boş görev eklemeyi engelleme
- `map` ile liste render + `key={todo.id}`
- `filter` ile silme (immutable güncelleme)
- `spread` ile `toggle({...todo, completed: !todo.completed})`
- `ternary` ile koşullu stil
- `&&` ile koşullu gösterim (özet satırı)

⚠️ Immutable Güncelleme

React'ta state **doğrudan değiştirilmez**. Diziden silmek için `splice` yerine `filter`, güncellemek için doğrudan atama yerine `map + spread` kullanırız. Bu sayede React değişiklikleri algılar ve ekranı günceller.

`todos.splice(i, 1)` ❌ Yanlış — orijinal diziyi değiştirir

`todos.filter(t => t.id !== id)` ✅ Doğru — yeni dizi döndürür

5 Özet ve Sonraki Session

5.1 Bu Session'da Öğrendiklerimiz

- ✓ Component lifecycle: mount, update, unmount
- ✓ `useEffect` ile yan etki yönetimi
- ✓ Dependency array: `[]`, `[a, b]`, hiç verilmezse ne olur
- ✓ Cleanup fonksiyonu ve memory leak önleme
- ✓ Conditional rendering: if/return, ternary (`? :`), `&&`
- ✓ Loading / Error / Data patterni
- ✓ List rendering: `map` ile dizi $\rightarrow$ UI
- ✓ `key` prop'unun önemi ve doğru kullanımı
- ✓ Form handling: `onSubmit`, `e.preventDefault()`
- ✓ Obje state ile çoklu input yönetimi
- ✓ Temel form validation
- ✓ Pratik: Mini ToDo uygulaması (CRUD)

5.2 Sıkça Sorulan Sorular

SSS

S: `useEffect` her render'da çalışır mı?

C: Dependency array'e bağlı. Boş dizi (`[]`) verilirse sadece mount'ta, belirli değerler verilirse o değerler değişince çalışır. Array hiç verilmezse her render'da çalışır — bu genellikle istenmez.

S: Neden `index` yerine `id` kullanmalıyız?

C: Liste sıralama, ekleme veya silme yapılırsa `index` değişir; React yanlış elemanları eşleştirir ve input state'leri kaybolabilir. Sabit bir `id` bu sorunu önler.

S: State'i neden `spread` ile güncelliyoruz?

C: React, referans karşılaştırması yapar. Aynı obje/diziyi mutate edersek React değişikliği fark edemez. `spread` ile yeni obje/dizi oluşturmak React'a "bu değişti, yeniden render et" sinyali verir.

S: Form validation için kütüphane kullanmalı mıyız?

C: Basit formlarda el ile yeterlidir. Karmaşık formlar için `react-hook-form` veya `Formik` gibi kütüphaneler hayatı kolaylaştırır — bunları ileride göreceğiz.

5.3 Sonraki Session — Session 5: React Best Practices

→ Session 5 (7 Mayıs Perşembe)

Kod kalitesini artırmak ve doğru mimari yaklaşımı öğrenmek için:

- **Component parçalama** ve reusable component mantığı
- **State yönetimi** yaklaşımları: lifting state up, props drilling
- **Custom hooks** ile mantık paylaşımı
- **Pratik:** ToDo uygulamasını refactor etme